

2024 Sep Oppe 1 - Set 1

Section 1 - problem 2

Is Odd-Length Palindrome

Write a function

`is_odd_length_palindrome` that takes a string `s` and returns **True** if `s` is a palindrome of odd length, and **False** otherwise.

****NOTE:** This is a function type question, you don't have to take input or print the output, just have to complete the required function definition.**

Example

Input: hello

Output: False

Reason: (not a palindrome) ❌

Input: noon

Output: False

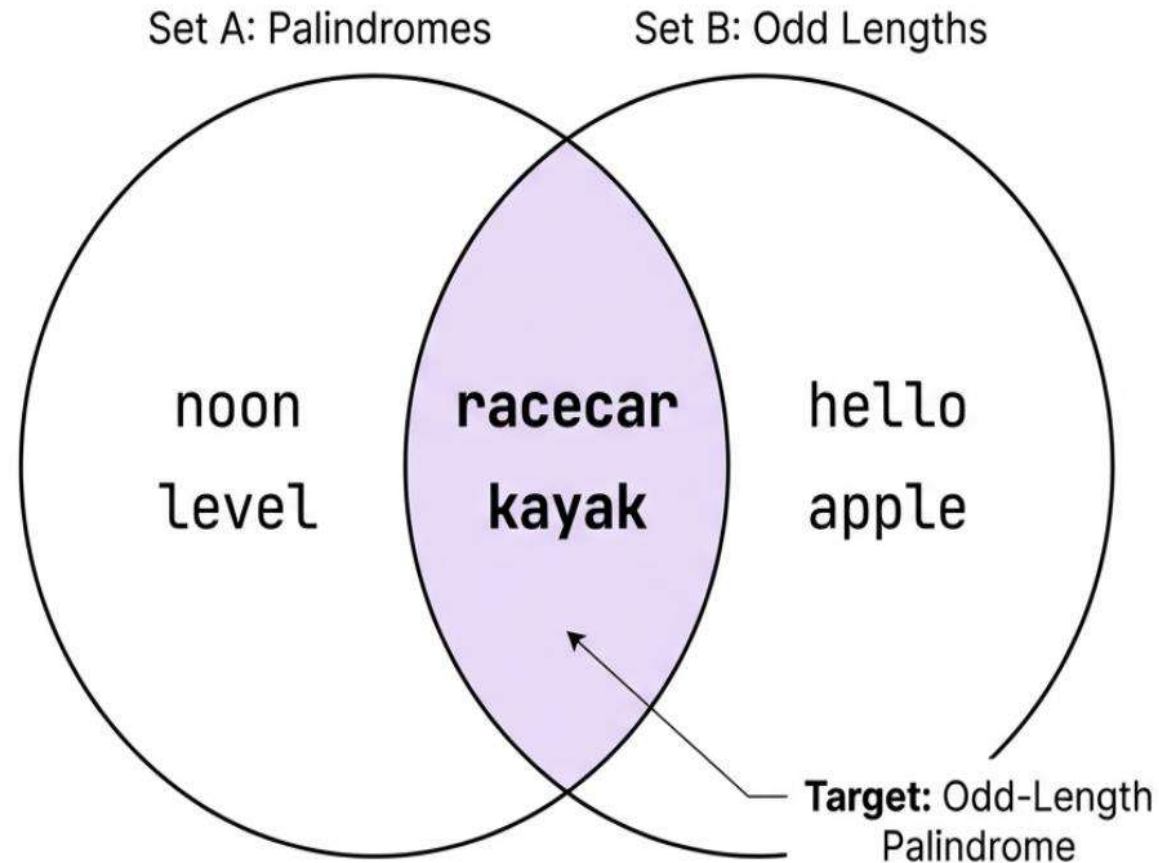
Reason: (palindrome but not odd length) ❌

Input: nun

Output: True

Reason: (palindrome and odd length) ✅

The Dual Constraint



Problem Definition

Input: `s`: string

Condition 1:

- **Symmetry:** The string must read the same forward and backward.

Condition 2:

- **Cardinality:** The character count must be an odd integer.

Validating the Conditions

Distinguishing false positives from true positives.

Input String	Condition A: Palindrome?	Condition B: Odd Length?	Final Result
"racecar"	Yes (reads as 'racecar')	Yes (7 characters)	TRUE
"noon"	Yes (reads as 'noon')	No (4 characters)	FALSE
"hello"	No (reads as 'olleh')	Yes (5 characters)	FALSE

Insight: A string can satisfy one condition but fail the function if the logical intersection is not met.

Visualizing the Mechanics

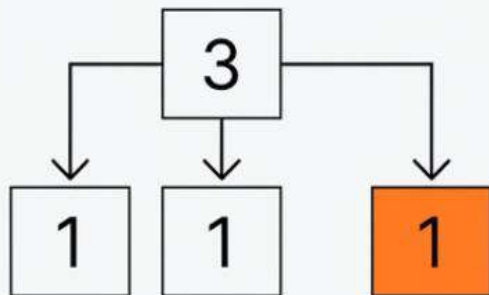
Operation: Reflection

aba

aba $\leftarrow s[: -1]$

Verifies symmetry by comparing the sequence against its reverse index.

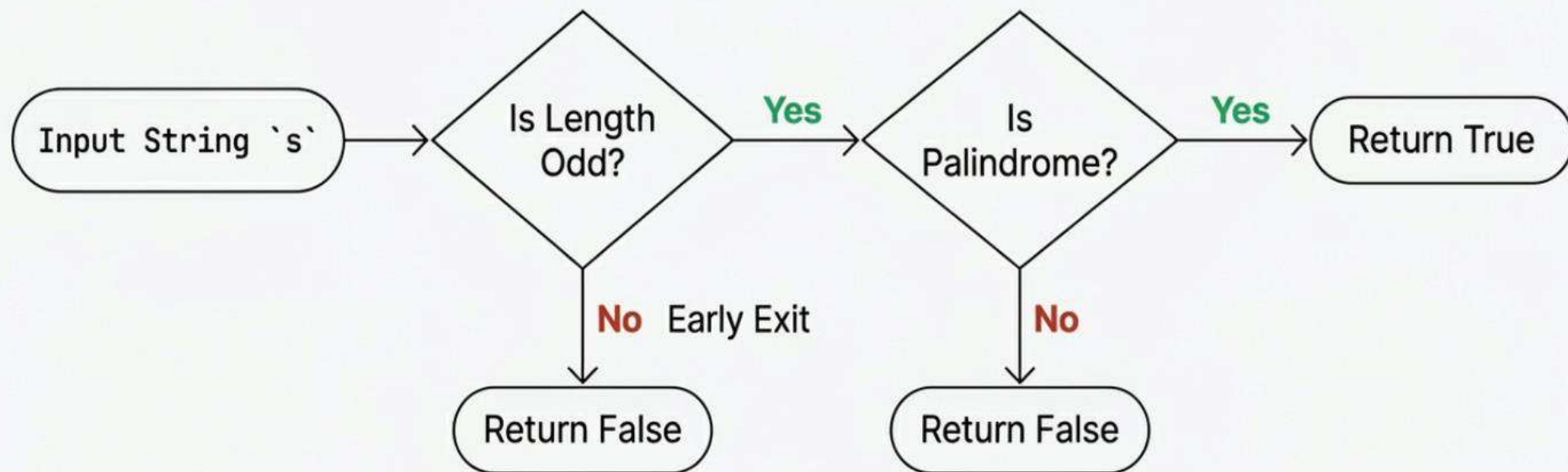
Operation: Modulo



$$3 \% 2 = 1$$

Verifies odd cardinality. A non-zero remainder indicates an odd number.

The Decision Tree

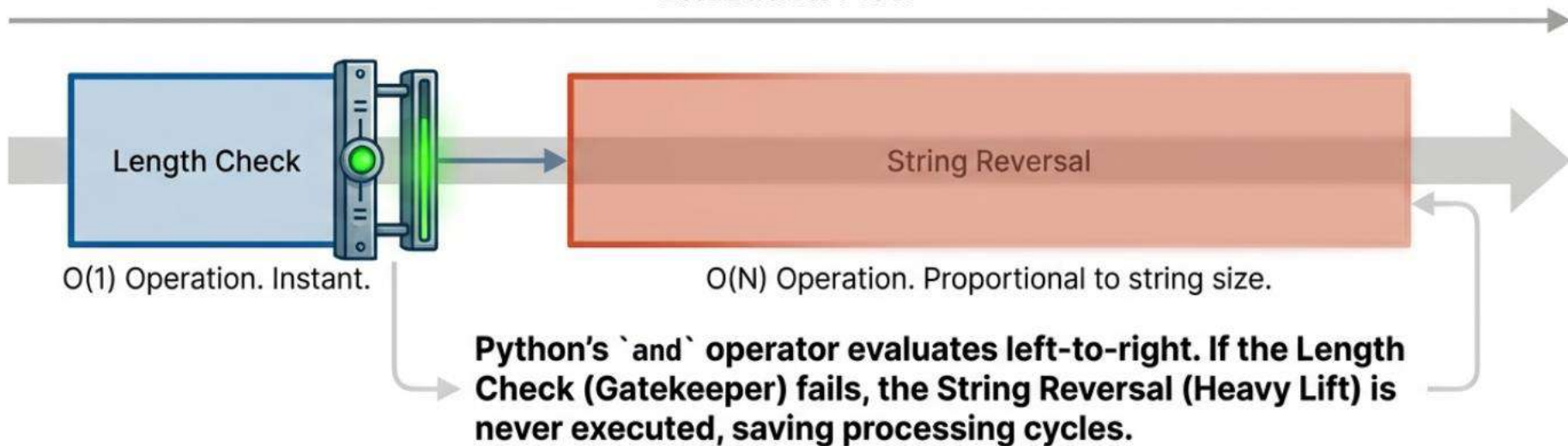


Sequential logic prevents unnecessary computation.

The Power of Short-Circuiting

Optimizing with the logical AND operator

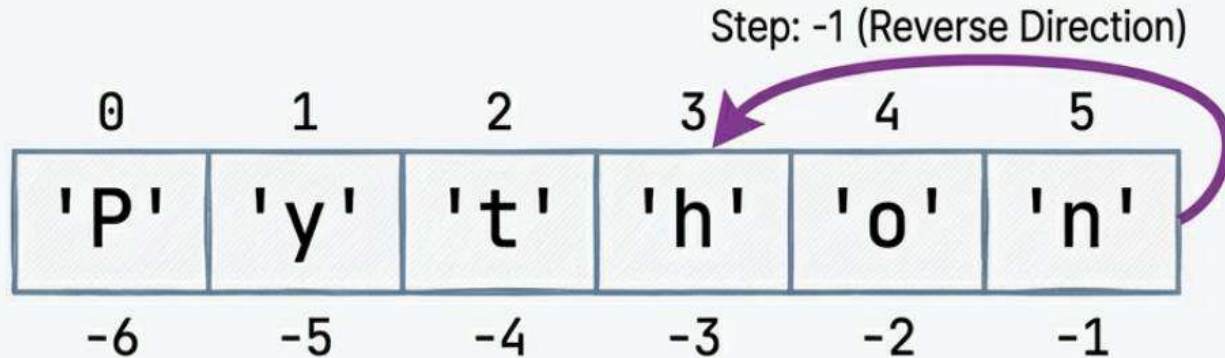
Execution Flow



```
if len(s) % 2 == 0:  
    return False
```

The Tool: Python Slicing

`s[::-1]`



Conciseness

Reverses a string without the boilerplate of a manual loop.

Readability

Recognized instantly by Python developers as the standard idiom.

Efficiency

Leverages highly optimized C-based internals.

The Elegant Solution

The Operator

Enables short-circuit evaluation.

```
def is_odd_length_palindrome(s: str) -> bool:  
    return len(s) % 2 != 0 and s == s[::-1]
```

The Gatekeeper

Checks for odd length first.
Fast O(1) check.

The Validator

Reverses and compares.
Only runs if length is odd.

Naive Approach vs. Pythonic Elegance

The Naive Approach

Verbose / Manual

```
def check_naive(s):  
    # Check length  
    if len(s) % 2 != 0:  
        return False  
  
    # Manual loop reversal  
    reversed_s = ""  
    for i in range(len(s)-1, -1, -1):  
        reversed_s += s[i]  
  
    if s == reversed_s:  
        return True  
    return False
```

High cognitive load. Manages indices manually.
More room for error.

The Pythonic Approach

Idiomatic / Concise

```
def check_pythonic(s):  
    return len(s) % 2 != 0 and s != s[::-1]
```

Self-documenting. Utilizes built-in language features. Logic is flat and linear.

Engineering Best Practices

1

Order Matters

Place computationally cheap checks (like Math) before expensive checks (like String Manipulation). This maximizes the benefit of short-circuiting.

2

Idiomatic Python

Prefer slicing (`[::-1]`) over manual loops. Writing 'Pythonic' code improves maintainability and signals expertise to other developers.

3

Boolean Simplification

Combine conditions with logical and operators rather than nesting multiple `if` statements. This flattens the cyclomatic complexity.

Logic Reference

Result = (Odd Length) AND (Palindrome)

```
return len(s) % 2 != 0 and s == s[::-1]
```

Efficiency is not just about speed; it is about doing only what is necessary.